23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : C15

Team Members :

Roll No	**Student Name**
AM.SC.U4CSE25251	Sravan N N
AM.SC.U4CSE25222	Harith Bhuvan S
AM.SC.U4CSE25213	Devang Dileep
AM.SC.U4CSE25233	Jasimkhan J L

Project Title : ShopSphere: A Relational Database Management System for a Multi-Vendor Online Shopping Platform

3.1. Detailed description of the project

ShopSphere is a centralized, database-driven e-commerce platform for multiple vendors, customers, platform administrators, and delivery personnel. It replaces fragmented spreadsheets, payment trackers, and manual order registers with one relational database that keeps catalogue, inventory, transaction, payment, and shipment information consistent.

Customers register with profiles and saved addresses, browse products across common categories and sub-categories, compare vendors and prices, add products from different vendors to one cart, apply discount coupons, complete checkout, and track orders. Vendors register and are approved before listing products, then manage products, inventory, fulfilment, and sales information. Administrators manage vendor verification, categories, and platform-level reporting, while delivery personnel update shipment status.

The core scenario is expressed through Customer, Address, Vendor, Admin, Category, Product, Inventory, Cart, Cart_Item, Coupon, Orders, Order_Item, Payment, Shipment, and Review. Foreign keys preserve referential integrity, and associative relations resolve many-to-many relationships such as Cart-Product and Order-Product.

Key business rules:

- A customer can place many orders, while each order belongs to exactly one customer.
- An order contains one or more order items; each item identifies one product and its fulfilling vendor.
- A vendor must be approved by an administrator before its products are listed.
- Each product belongs to one category; the category hierarchy supports sub-categories through a self-reference.

- Each product has one inventory record and stock is updated atomically during checkout.
- An order may use zero or one coupon; payment attempts are stored separately so failed/retry attempts can be recorded.
- A review is allowed only for a product that appears in a delivered order item of that customer.
- Order items can be shipped independently, allowing multi-vendor orders to be fulfilled separately.

3.2. ER Diagram

The ER diagram is intentionally simplified to the core entities so that the main business flow remains readable. Relationship diamonds, 1/M cardinality labels, primary keys, foreign keys, and mandatory/optional participation are retained; the legend and notes are placed below the figure.

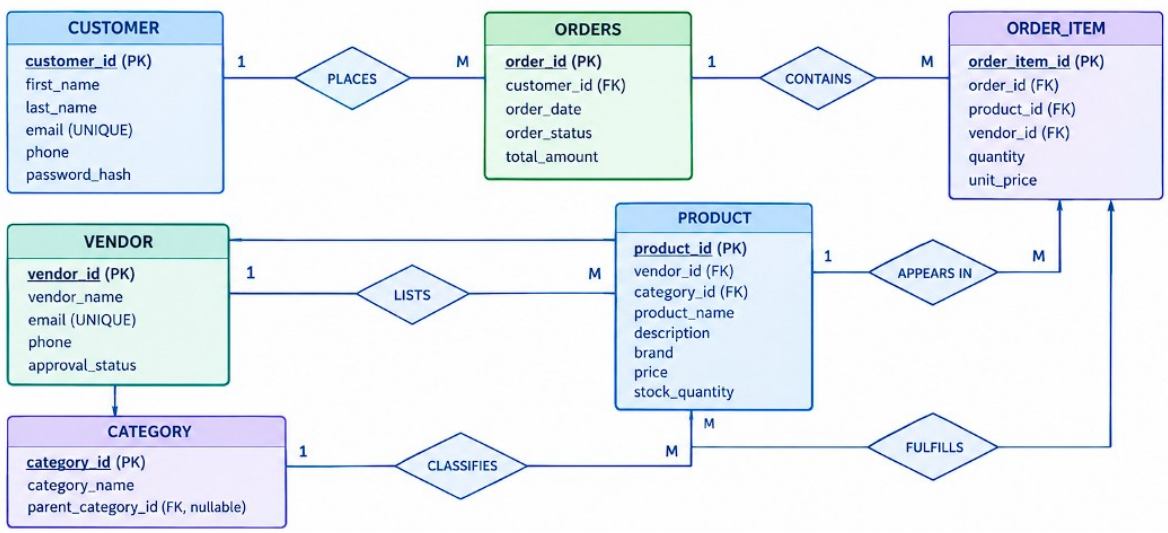


Figure 3.2: Simplified ShopSphere Entity-Relationship Diagram

Legend

- Entity: rectangular box representing an entity and its attributes.
- Relationship: diamond representing the semantic connection between entities.
- Solid line: mandatory (total) participation; dashed line: optional (partial) participation.
- 1: one occurrence; M/N: many occurrences; PK: primary key; FK: foreign key.

Diagram notes

- The simplified ER model focuses on the core customer, vendor, product catalogue, and order-processing flow of ShopSphere.
- Category contains parent_category_id for the recursive sub-category relationship.
- Order_Item acts as the associative relation for an order containing products, while the vendor_id identifies the fulfilling vendor.

- Additional supporting relations such as Address, Cart, Coupon, Payment, Shipment, Inventory, and Review are retained in the full relational schema in Section 3.3.

Relationship interpretation

- Customer - Orders (1:M): one customer may place many orders.
- Orders - Order_Item (1:M): an order is decomposed into line items.
- Product - Order_Item (1:M): a product may occur in many order lines.
- Vendor - Product (1:M): one vendor can list many products.
- Category - Product (1:M): one category can classify many products.
- Product - Category and Vendor links are carried into the relational schema as foreign keys.
- The ER diagram shows the simplified conceptual core; the remaining operational details are represented in the relational schema.

3.3 Relational Schema

The ER model is translated into relations using primary keys, foreign keys, unique constraints, and associative relations. To keep the document structured and readable, the fifteen final relations are grouped into five schema tables rather than placing every relation in a separate table.

Mapping notes: Product-Inventory is implemented as a one-to-one relationship by using product_id as both PK and FK in Inventory. Cart_Item resolves Cart-Product, while Order_Item records the product and fulfilling vendor for each order line. The recursive Category relationship uses nullable parent_category_id. The UNIQUE constraint on Shipment.order_item_id ensures one shipment per order item.

Schema Group 1 - Users, Addresses and Vendors	
Customer	customer_id (PK), first_name, last_name, email (UNIQUE), phone, password_hash, registration_date
Address	address_id (PK), customer_id (FK -> Customer), address_line1, address_line2, city, state, pincode, address_type
Admin	admin_id (PK), name, email (UNIQUE), password_hash
Vendor	vendor_id (PK), vendor_name, email (UNIQUE), phone, business_license_no, approval_status, registration_date

Schema Group 2 - Catalogue and Inventory	
Category	category_id (PK), category_name, parent_category_id (FK -> Category, nullable)
Product	product_id (PK), vendor_id (FK -> Vendor), category_id (FK -> Category), product_name, description, brand, price, created_date
Inventory	product_id (PK, FK -> Product), stock_quantity, last_updated

Schema Group 3 - Cart and Discounting	
Cart	cart_id (PK), customer_id (FK -> Customer, UNIQUE)
Cart_Item	cart_id (PK1, FK -> Cart), product_id (PK2, FK -> Product), quantity
Coupon	coupon_id (PK), coupon_code (UNIQUE), discount_percent, valid_from, valid_to, min_order_value, usage_limit

Schema Group 4 - Orders and Line Items	
Orders	order_id (PK), customer_id (FK -> Customer), coupon_id (FK -> Coupon, nullable), order_date, order_status, total_amount
Order_Item	order_item_id (PK), order_id (FK -> Orders), product_id (FK -> Product), vendor_id (FK -> Vendor), quantity, unit_price

Schema Group 5 - Payments, Shipments and Reviews	
Payment	payment_id (PK), order_id (FK -> Orders), payment_mode, amount, payment_status, payment_date
Shipment	shipment_id (PK), order_item_id (FK -> Order_Item, UNIQUE), courier_name, tracking_number, shipment_status, shipped_date, delivered_date
Review	review_id (PK), customer_id (FK -> Customer), product_id (FK -> Product), rating, comment, review_date

3.4 Normalization and Functional dependency analysis

A flat OrderDetails relation is used to demonstrate the normalization process required by the design form. The relation contains order_id, order_date, customer_id, customer_name, customer_email, product_id, product_name, category_id, category_name, vendor_id, vendor_name, quantity, unit_price, coupon_code, and discount_percent.

3.4.1 Functional dependencies

- F1: order_id -> order_date, customer_id, coupon_code
- F2: customer_id -> customer_name, customer_email
- F3: product_id -> product_name, category_id, vendor_id
- F4: category_id -> category_name
- F5: vendor_id -> vendor_name
- F6: coupon_code -> discount_percent
- F7: (order_id, product_id) -> quantity, unit_price

These dependencies show that order-level values depend on order_id, product-level values depend on product_id, and line-level values depend on the combination

(order_id, product_id). Keeping all attributes in one relation would therefore introduce redundancy and update anomalies.

3.4.2 Candidate key - attribute closure

Claim: {order_id, product_id} is a candidate key of OrderDetails. Starting from {order_id, product_id}, F1 adds order_date, customer_id, and coupon_code; F3 adds product_name, category_id, and vendor_id; F2 adds customer_name and customer_email; F4 adds category_name; F5 adds vendor_name; F6 adds discount_percent; and F7 adds quantity and unit_price. The closure therefore contains all 15 attributes.

Minimality holds because order_id alone cannot determine product attributes and product_id alone cannot determine order attributes. Thus neither component is a key by itself, but the combination determines the full relation, so {order_id, product_id} is the minimal candidate key.

3.4.3 Normalization: 1NF -> 2NF -> 3NF -> BCNF

1NF: OrderDetails is in 1NF because each field is atomic and every row represents one order line without repeating groups.

2NF: Because the key is composite, F1 and F3 are partial dependencies on order_id and product_id separately. Decomposition removes these dependencies into OrderInfo(order_id, order_date, customer_id, customer_name, customer_email, coupon_code, discount_percent), ProductInfo(product_id, product_name, category_id, category_name, vendor_id, vendor_name), and OrderItem(order_id, product_id, quantity, unit_price).

3NF: OrderInfo has transitive dependencies order_id -> customer_id -> customer_name/customer_email and order_id -> coupon_code -> discount_percent. ProductInfo has product_id -> category_id -> category_name and product_id -> vendor_id -> vendor_name. Further decomposition yields Orders, Customer, Coupon, Product, Category, Vendor, and OrderItem.

BCNF: in each resulting relation, every non-trivial determinant is a candidate key of that relation. Orders, Customer, Coupon, Product, Category, and Vendor each have a determinant equal to the primary key, while OrderItem has determinant (order_id, product_id), equal to its key. Hence these relations satisfy BCNF.

3.4.4 Final normalization summary

The remaining relations - Address, Admin, Inventory, Cart, Cart_Item, Payment, Shipment, and Review - follow the same principle: non-key attributes depend directly on their keys, without partial or transitive dependency, and no non-key determinant is introduced. The full ShopSphere schema therefore satisfies the requested normalization target of BCNF.